Reg. No

SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA
(UNIVERSITY ESTABLISHED under section 3 of UGC Act 1956)

ENATHUR, KANCHIPURAM – 631 561

DATA STRUCTURES AND ALGORITHMS
LABORATORY RECORD

Name :G Gokul

REG No : 112534074

Class : I YEAR BCA

Subject : UCAF252T40 – DSA Lab

SRI CHANDRASEKHARENDRA SARASWATHI
VISWA MAHAVIDYALAYA
(University Established under section 3 of UGC Act 1956)

BONAFIDE CERTIFICATE

This is to Certify that this is the bonafide record of work done

by _____

_____

Mr. G Gokul with Reg. No 112534074 of I Year BCA

inthe Data Structures and Algorithms Lab

during the year 2026.

Staff-in-charge Head of the Department

Submitted for the Practical Examination held on _____

Internal Examiner External Examiner.

INDEX

S.No.	Date	Title	Page No.	Signature
1	06 Feb 2026	Program to Search an Element in the Array.	01	
2	13 Feb 2026	Program to Search an Element In the Array using Binary Search.	05	
3	20 Feb 2026	Program to Sort an Array using Bubble Sort.	10	
4	27 Feb 2026	Program to Sort an Array using Selection Sort.	13	
5	06 Feb 2026	Program to Sort an Array using Insertion Sort.	17	
6	13 Mar 2026	Program to Implement a Stack Using Arrays.	20	
7	20 Mar 2026	Program to Implement a Queue Using Arrays	26	

8	27 Mar 2026	Program to Implement a Stack using Linked List	33	
9	03 Apr 2026	Program to Implement a Queue using Linked List	38	

1	Program to Search an Element In the Array	06 Feb 2026

AIM:

Program to Search an Element in the Array using Linear Search.

ALGORITHM:

Step 1: Start.

Step 2: Declare the counter variable (i), array size (n), element to search found.

Step 3: Read = n, i = 0

Step 4: Read arr[i]

Step 5: increment i (i++)

Step 6: if (i< n) goto step 4.

Step 7: Read element to search (search).

Step 8: i = 0

Step 9: if arr[i] ! = Search goto step 12.

Step 10: found = 1.

Step 11: print “Element found at index = i” go to step 15.

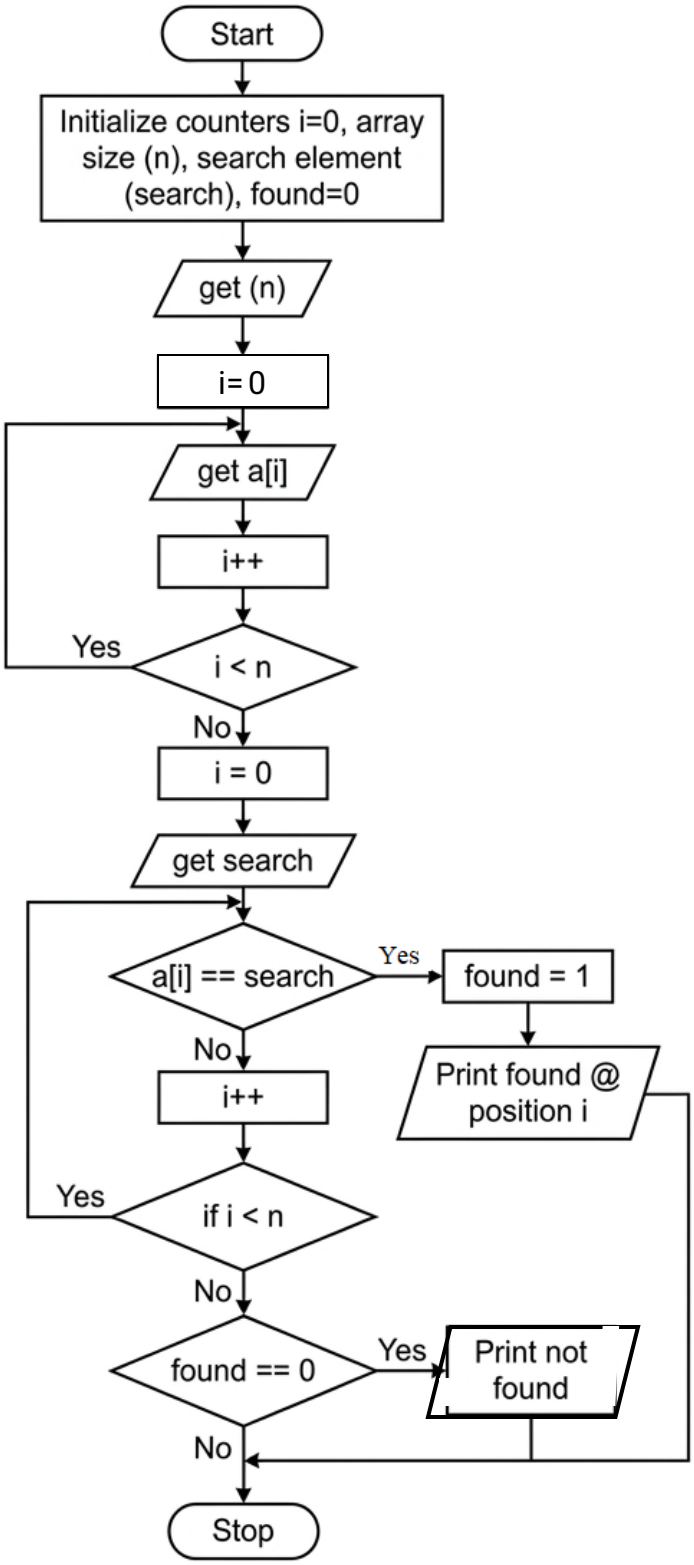
Step 12: i++

Step 13: if i < n go to step 9.

Step 14: if found not equal to 1, print “Element not found”.

Step 15: stop.

FLOW CHART:


Start
Initialize counters i=0, array size (n), search element (search), found=0
get (n)
i= 0
get a[i]
i++
Yes
i < n
No
i = 0
get search
a[i] == search
Yes
found = 1
Print found @ position i
No
i++
Yes
if i < n
No
found == 0
Yes
Print not found
No
Stop

SOURCE CODE:

https://github.com/112534080-stack/DSALab/blob/main/ArrayStack.c

```
#include <stdio.h>
int main()
{
    int n, i, search, found = 0;
    // Input number of elements in the array
    printf("Enter number of elements in the array: ");
    scanf("%d", &n);
    int arr[n];
    // Input array elements
    printf("Enter elements of the array: \n");
    for(i= 0; i< n; i++) {
    scanf("%d", &arr[i]);
    }
    // Input the element to search
    printf("Enter element to search: ");
    scanf("%d", &search);
    // Linear search
    for(i = 0; i< n; i++)
{
    if(arr[i] == search)
   {
    found = 1;
     printf("Element %d found at index %d.\n", search, i);
     break;
     }
    }
    // If element is not found
    if(!found)
   {
      printf("Element %d not found in the array.\n", search);
    }
    return 0;
}
```

SAMPLEOUTPUT:

```
Enter number of elements in the array: 5
Enter elements of the array:
6
3
8
5
2
Enter element to search: 8
Element 8 found at index 2.
NEGATIVE I/O
Enter number of elements in the array: 5
Enter elements of the array:
6
3
8
5
2
Enter element to search: 10
Element 10 not found in the array
```

2	Program to Search an Element In the Array using Binary Search	13 Feb 2026

AIM:

Program to Search an Element in the Array using Binary Search.

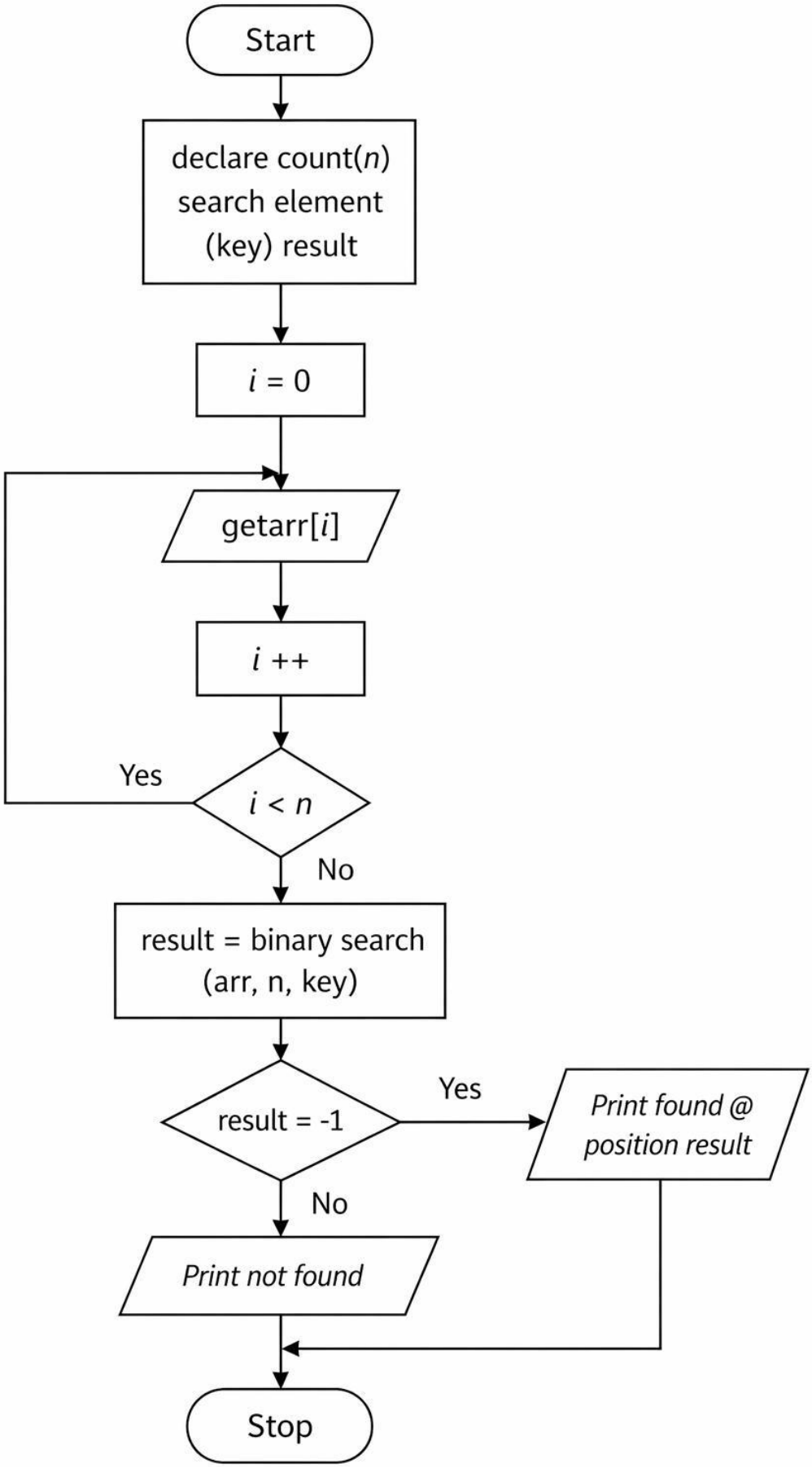
ALGORITHM:

Step 1: start
Step 2: declare no of elements (n) elements to search (key)
Step 3: initialization array with n(arr[n])
Step 4: i=0
Step 5: get arr[i]
Step 6: i++
Step 7: if i< n goto step 5,else continue
Step 8: read element to search(key)
Step 9: result = binary search(arr,n,key)
Step 10: if (result!=1)
Step 11: print element found at position(result)else
Print element not found
Step 12: stop

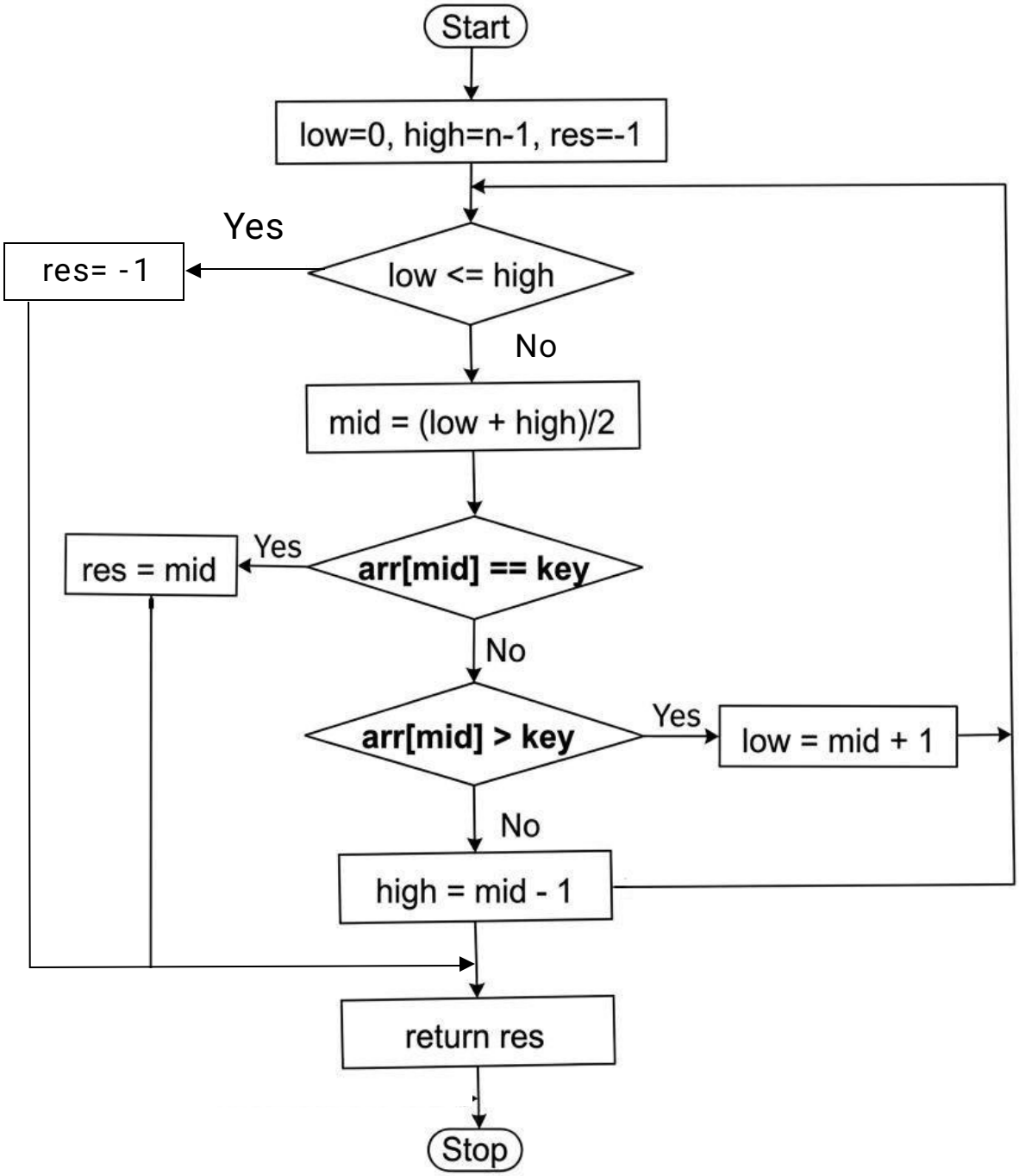
ALGORITHM FOR BINARY SEARCH:

```
Step 1: int low=o, high=n-1
Step 2: if low(<=high)
Step 3: mid=(low+high)/2
Step 4: if (arr[mid]==key)
        Return mid
Step 5: else if (arr[mid]<key)
        Low=mid+1
        Else
          High+mid-1
Step 6: goto step 2
Step 7: return-1
```

FLOW CHART:


Start
declare count(n)
search element
(key) result
i = 0
getarr[i]
i ++
Yes
i < n
No
result = binary search
(arr, n, key)
result = -1
Yes
Print found @
position result
No
Print not found
Stop

Binarysearch(int arr[ ], int n, int key)



SOURCE CODE:

https://github.com/112534080-stack/DSALab/blob/main/Binarysearch.c

```
#include< stdio.h>
int binarysearch(int arr[],int n, int key)
{
int low= 0,high= n- 1,mid;
while(low< =high)
{
mid= (low+high)/2;
if (arr[mid]= =key)
return mid;
//element found
else if(arr[mid]< key)
low= mid+ 1;
//search right half
else
high= mid- 1;
//search left half
}
return 1;
//element not found
}
int main()
{
int n,key,result;
```

```
printf("enter number of element:");
scanf("% d",&n);
int arr[n];
printf("enter % d sorted element:\n",n);
int i;
for(i=0;i<n;i++){
scanf("% d",&arr[i]);
}
printf("enter element to search:");
scanf("% d",&key);
result = binarysearch(arr, n, key);
if(result!=-1)
printf("element found at index % d \n",result);
else
printf("element not found \n");
return 0;
}
```

SAMPLE OUTPUT:

```
Enter number of elements: 5
Enter 5 sorted elements:
11
12
13
14
15
Enter element to search: 14
Element found at index 3
```

3	Program to Sort an Array using Bubble Sort	20 Feb 2026

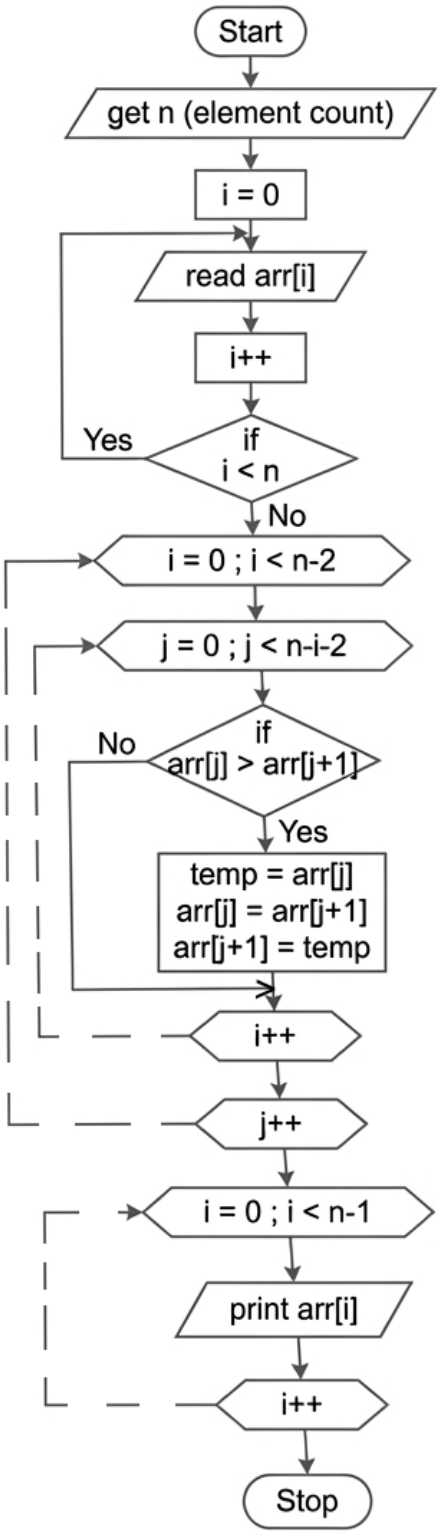
AIM:

Program to Sort an Array using Bubble Sort.

ALGORITHM:

Step 1: Start the program
Step 2: Read the number of elemnt n
Step 3: Declare an array arr[n]
Step 4: Input n elements into the array
Step 5: for i=0 to n-2
Step 6: for j=0 to n-i-2
Step 7: if arr[j]>arr[j+1]
Swap arr[j] and arr[j+1]
Temp=arr[j]
Arr[j]=arr[j+1]
Arr[j+1]=temp
Step 8: Print the sorted array
Step 9: Stop

FLOW CHART:


Start
get n (element count)
i = 0
read arr[i]
i++
Yes
if
i < n
No
i = 0 ; i < n-2
j = 0 ; j < n-i-2
No
if
arr[j] > arr[j+1]
Yes
temp = arr[j]
arr[j] = arr[j+1]
arr[j+1] = temp
i++
j++
i = 0 ; i < n-1
print arr[i]
i++
Stop

SOURCE CODE :

https://github.com/112534080-stack/DSALab/blob/main/BubbleSort.c

```
#include< stdio.h>
int main()
{
int n,temp;
int i,j;
printf("enter number of element:");
scanf("% d",&n);
int arr[n];
printf("enter % d elements:\n",n);
for(i= 0;i< n;i++)
scanf("% d",&arr[i]);
//bubble sort logic
for(i= 0;i< n;i++)
{
for(j= 0;j< n;j++)
{
if(arr[j]> arr[j+ 1])
{
temp= arr[j];
arr[j]= arr[j+ 1];
arr[j+ 1]= temp;
}
}
}
printf("sorted array:\n");
for(i= 0;i< n;i++)
printf("% d",arr[i]);
return 0;
}
```

SAMPLE OUTPUT:

```
Enter number of element:5
        Enter 5 element:
```

25

32

95

65

23

Sorted array

23 25 32 65 95

4	Program to Sort an Array using Selection Sort	27 Feb 2026

AIM:

Program to Sort an Array using Selection Sort.

ALGORITHM:

Step 1: start

Step 2: declare element count (n),

Step 3: get n

Step 4: declare arr [n]

Step 5: get arr [i]

Step 6: i++

Step 7: repeat from step 5 till i< n

Step 8: I = 0

Step 9 : minindex = i

Step 10: j = i+1

Step 11: if arr [i]< arr

Step 12: temp = arr [i]

arr[i] = arr[minindex]

arr[minindex] = temp

Step 13: j++

Step 14: repeat from step 11 till j< n

Step 15: i++

Step 16: repeat from step 9 till i< n-1

Step 17: stop I = 0

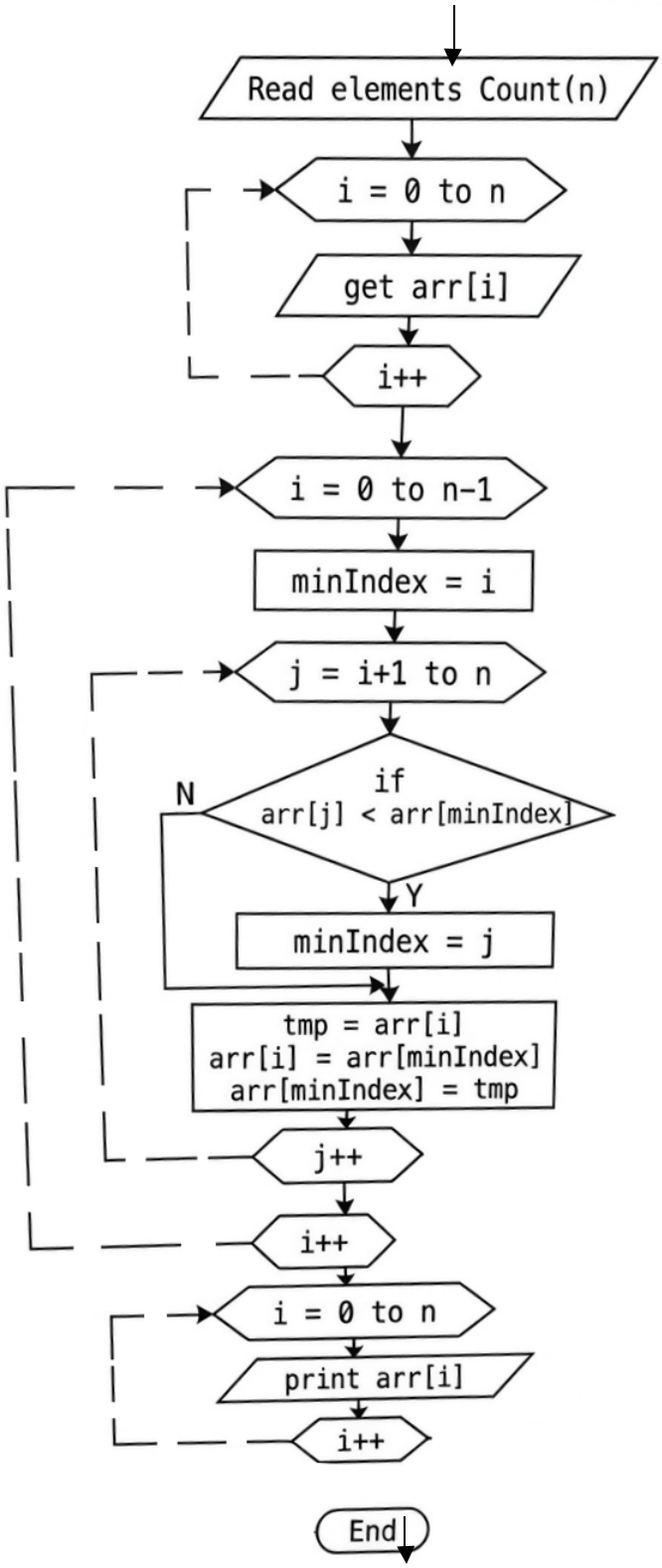
Step 18: print arr [i]

Step 19: i++

Step 20: repeat from step 18 till i< n

Step 21: stop

FLOW CHART:


Read elements Count(n)
i = 0 to n
get arr[i]
i++
i = 0 to n-1
minIndex = i
j = i+1 to n
if
arr[j] < arr[minIndex]
N
Y
minIndex = j
tmp = arr[i]
arr[i] = arr[minIndex]
arr[minIndex] = tmp
j++
i++
i = 0 to n
print arr[i]
i++
End

SOURCE CODE:

https://github.com/112534080-stack/DSALab/blob/main/SelectionSort.c

```
#include <stdio.h>
int main()
{
    int n, temp, minindex;
    printf("enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("enter %d elements:\n", n);
    for (int i = 0; i < n; i++)
    scanf("%d", &arr[i]);
    // selection sort logic
    for (int i = 0; i < n – 1; i++)
    {
        minindex = i;
        for (int j = i + 1; j < n; j++)
        {
            if (arr[j] < arr[minindex])
            minindex = j;
        }
        // swap
        temp = arr[i];
        arr[i] = arr[minindex];
        arr[minindex] = temp;
    }
    printf("sorted array:\n");
    for (int i = 0; i < n; i++)
```

```
    printf("% d ", arr[i]);
    return 0;
}
```

OUTPUT:

```
Enter number of elements: 6
Enter 6 elements:
12
14
11
22
20
19
Sorted array:
11 12 14 19 20 22
```

5	Program to Sort an Array using Insertion Sort.	06 Feb 2026

AIM:

Program to Sort an Array using Insertion Sort.

ALGORITHM:

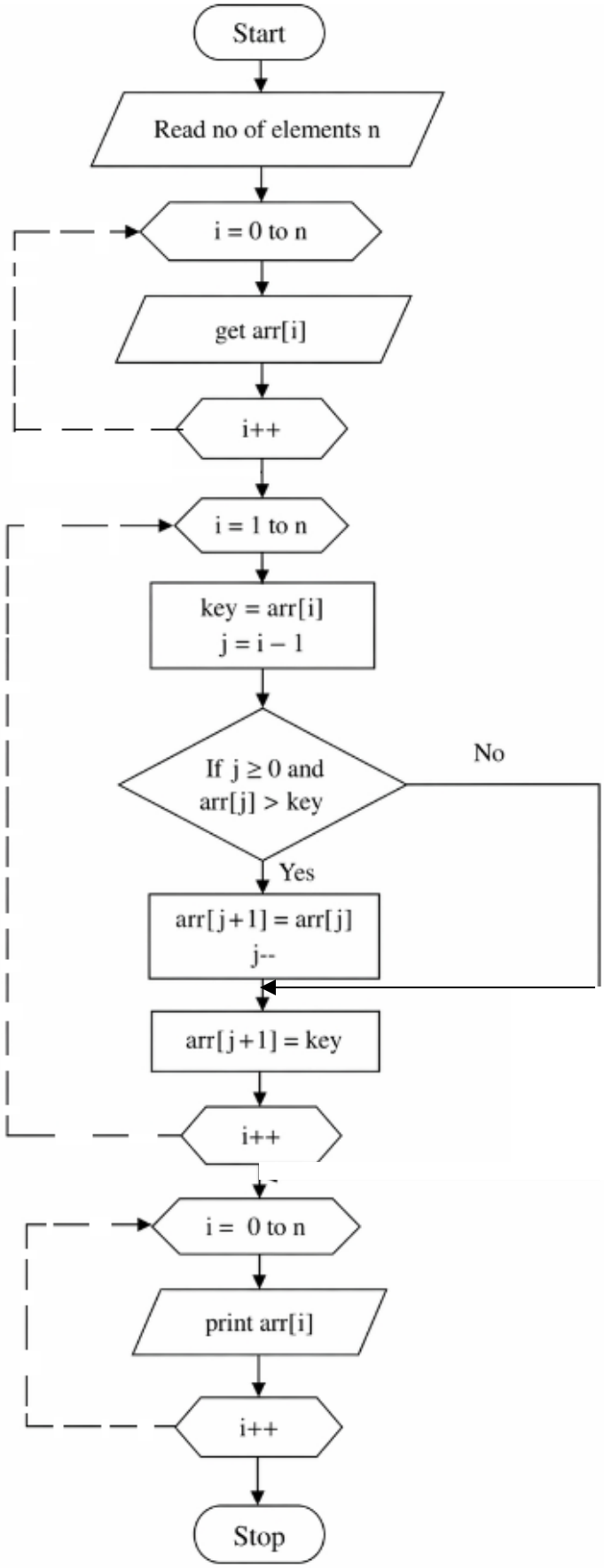
Step 1: Start
Step 2: Read the number of elements (n)
Step 3: Declare an array of size n, arr[n]
Step 4: Read array elements iteratively
for i = 0 to n-1 do:
- Prompt user: "Enter element i+1"
- Read arr[i] from user input

Step 5: Sort the array using Insertion Sort
for i = 1 to n-1 do:
- key = arr[i]
- j = i – 1
- Compare and shift elements:
- while j >= 0 and arr[j] > key do:
  - move arr[j] one position right: arr[j + 1] = arr[j]
  - j = j – 1
  - insert the key at its correct position:
    - arr[j + 1] = key

- repeat for the next element (i = i + 1)

Step 6: Display the sorted array
for i = 0 to n-1 do:
print arr[i]
Step 7: End

FLOW CHART:


Start
Read no of elements n
i = 0 to n
get arr[i]
i++
i = 1 to n
key = arr[i]
j = i – 1
If j ≥ 0 and
arr[j] > key
No
Yes
arr[j+1] = arr[j]
j--
arr[j+1] = key
i++
i = 0 to n
print arr[i]
i++
Stop

SOURCE CODE:

https://github.com/112534080-stack/DSALab/blob/main/Insertion%20Sort.c

```
#include <stdio.h>
int main()
{
    int n, key, j;
    printf("enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("enter %d elements:\n", n);
    for (int i = 0; i < n; i++)
    scanf("%d", &arr[i]);
    // insertion sort logic
    for (int i = 1; i < n; i++)
    {
        key = arr[i];
        j = i - 1;
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
    printf("sorted array:\n");
    for (int i = 0; i < n; i++)
     printf("%d ", arr[i]);
    return 0;
}
```

SAMPLE OUTPUT:

```
Enter number of elements: 6
Enter 6 elements:
12
14
11
22
```

```
20
19
Sorted array:
11 12 14 19 20 22
```

6	Program to Implement a Stack Using Arrays	13 March 2026

AIM:

Program to Implement a Stack Using Arrays.

ALGORITHM:

Step 1: Start – Begin program execution.
Step 2: Initialize Stack – Define a stack of size MAX (5).
Step 3: Initialize top – Set top = -1 to indicate the stack is empty.
Step 4: Display Main Menu – Show options: 1. Push 2. Pop 3. Display 4. Exit.
Step 5: Read User Choice – Read the choice from the user.
If choice = 1, go to Push Operation Steps.
If choice = 2, go to Pop Operation Steps.
If choice = 3, go to Display Operation Steps.
If choice = 4, go to Exit Operation Steps.
Otherwise, print "Invalid choice" and return to Step 4 (Display Main Menu).

Push ( ) Operation Steps (Choice = 1)
Step P1: Check Stack Overflow – If top == MAX – 1, print "Stack Overflow" and return to step 4 (Display MainMenu).
Step P2: Read Item – Read the element to push from the user.
Step P3: Increment top – top = top + 1.
Step P4: Assign Stack Element – stack[top] = item.
Step P5: Print Success Message – Print "Element pushed successfully" and return to Step 4 (Display Main Menu).

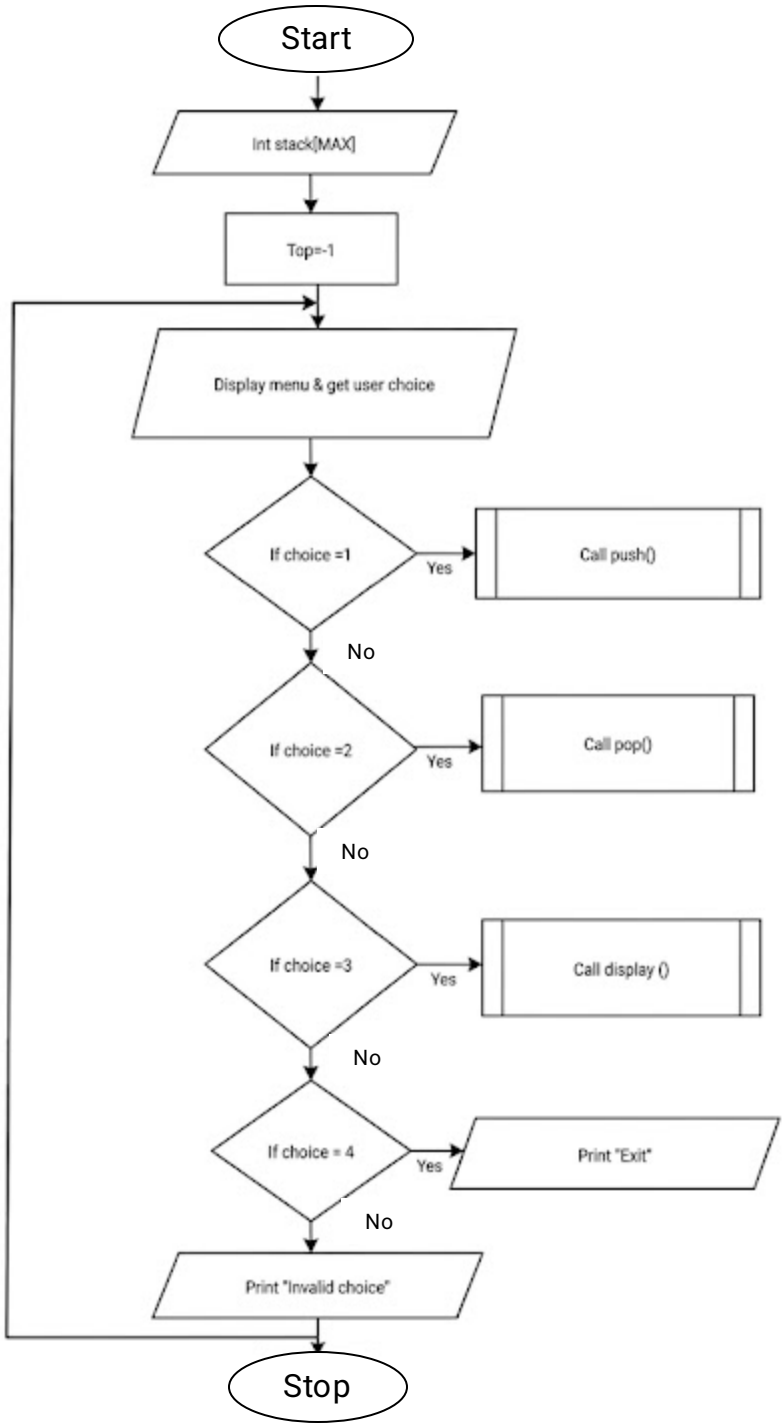
Pop( ) Operation Steps (Choice = 2)
Step O1: Check Stack Underflow – If top == -1, print "Stack Underflow" and return toStep4 (Display Main Menu).
Step O2: Print Popped Element – Print stack[top].
Step O3: Decrement top – top = top – 1 and return to Step 4 (Display Main Menu).

Display( ) Operation Steps (Choice = 3)
Step D1: Check if Stack is Empty – If top == -1, print "Stack is empty" and return to Step 4 (Display Main Menu).
Step D2: Loop Through Stack – For I = top down to 0, print stack[i].
Step D3: Return to Main Menu – Go back to Step 4 (Display Main Menu).

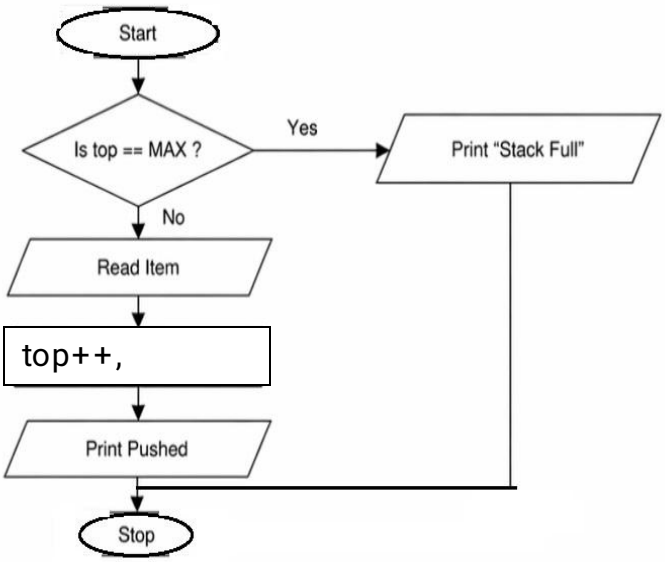
Exit( ) Operation Steps (Choice = 4)
Step E1: Print Exit Message – Print "Exiting program".

Step E2: End – Program execution ends.

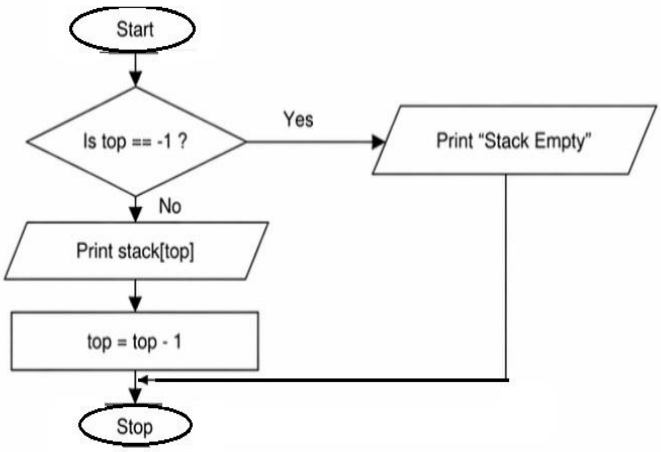
FLOWCHART:


Start
Int stack[MAX]
Top=-1
Display menu & get user choice
If choice =1
Yes
Call push()
No
If choice =2
Yes
Call pop()
No
If choice =3
Yes
Call display ()
No
If choice = 4
Yes
Print "Exit"
No
Print "Invalid choice"
Stop

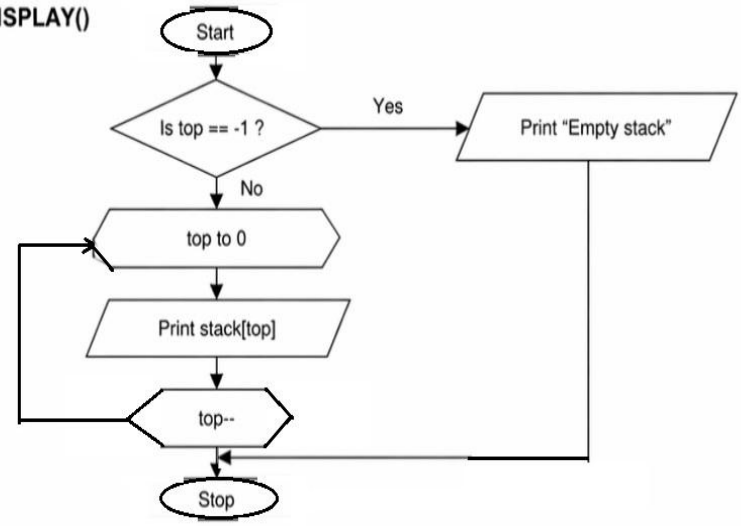
1. PUSH()



2. POP()



3. DISPLAY()



SOURCE CODE:

https://github.com/112534080-stack/DSALab/blob/main/ArrayStack.c

```
#include < stdio.h>
#define max
int stack[max];
int top = -1;
// push operation
void push()
{
    int item;
    if (top == max – 1)
    {
        printf("stack overflow!\n");
        return;
    }
    printf("enter element to push: ");
    scanf("% d", &item);
    stack[++top] = item;
    printf("element pushed successfully.\n");
}
// pop operation
void pop()
{
    if (top == -1)
    {
        printf("stack underflow!\n");
        return;
    }
    printf("popped element: % d\n", stack[top--]);
}
// display stack elements
void display()
{
    if (top == -1)
    {
        printf("stack is empty.\n");
        return;
    }
    printf("stack elements:\n");
```

```
    for (int i = top; i >= 0; i--)
        printf("% d\n", stack[i]);
}
int main()
{
    int choice;
    do
    {
        print("\n--- stack menu ---\n");
        printf("1. push\n2. pop\n3. display\n4. exit\n");
        printf("enter your choice: ");
        scanf("% d", &choice);
        switch (choice)
        {
        case 1:
            push();
            break;
        case 2:
            pop();
            break;
        case 3:
            display();
            break;
        case 4:
            printf("exiting program.\n");
            break;
        default:
            printf("invalid choice!\n");
        }
    } while (choice != 4);

    return 0;
}
```

SAMPLE OUTPUT:

```
--- Stack Menu ---
1. Push
2. Pop
3. Display
```

```
4. Exit
Enter your choice: 1
Enter element to push: 12
Element pushed successfully.

--- Stack Menu ---
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter element to push: 13
Element pushed successfully.
--- Stack Menu ---
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Stack elements:
13
12

--- Stack Menu ---
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 2
Popped element: 13

--- Stack Menu ---
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Stack elements:
12
```

--- Stack Menu ---
1. Push
2. Pop
3. Display
4. Exit
Enter your choice:
4
Exiting program.

7	Program to Implement a Queue Using Arrays	20 March 2026

AIM:

Program to Implement a Queue Using Arrays

ALGORITHM:

Step 1: Start – Begin program execution.
Step 2: Initialize Queue – Define a queue of size MAX (5).
Step 3: Initialize front and rear – Set front = -1 and rear = -1 to indicate the queue is empty.
Step 4: Display Main Menu – Show options: 1. Enqueue 2. Dequeue 3. Display 4. Exit.
Step 5: Read User Choice – Read the choice from the user.
If choice = 1, go to Enqueue Operation Steps.
If choice = 2, go to Dequeue Operation Steps.
If choice = 3, go to Display Operation Steps.
If choice = 4, go to Exit Operation Steps.
Otherwise, print “Invalid choice” and return to Step 4 (Display Main

Menu).

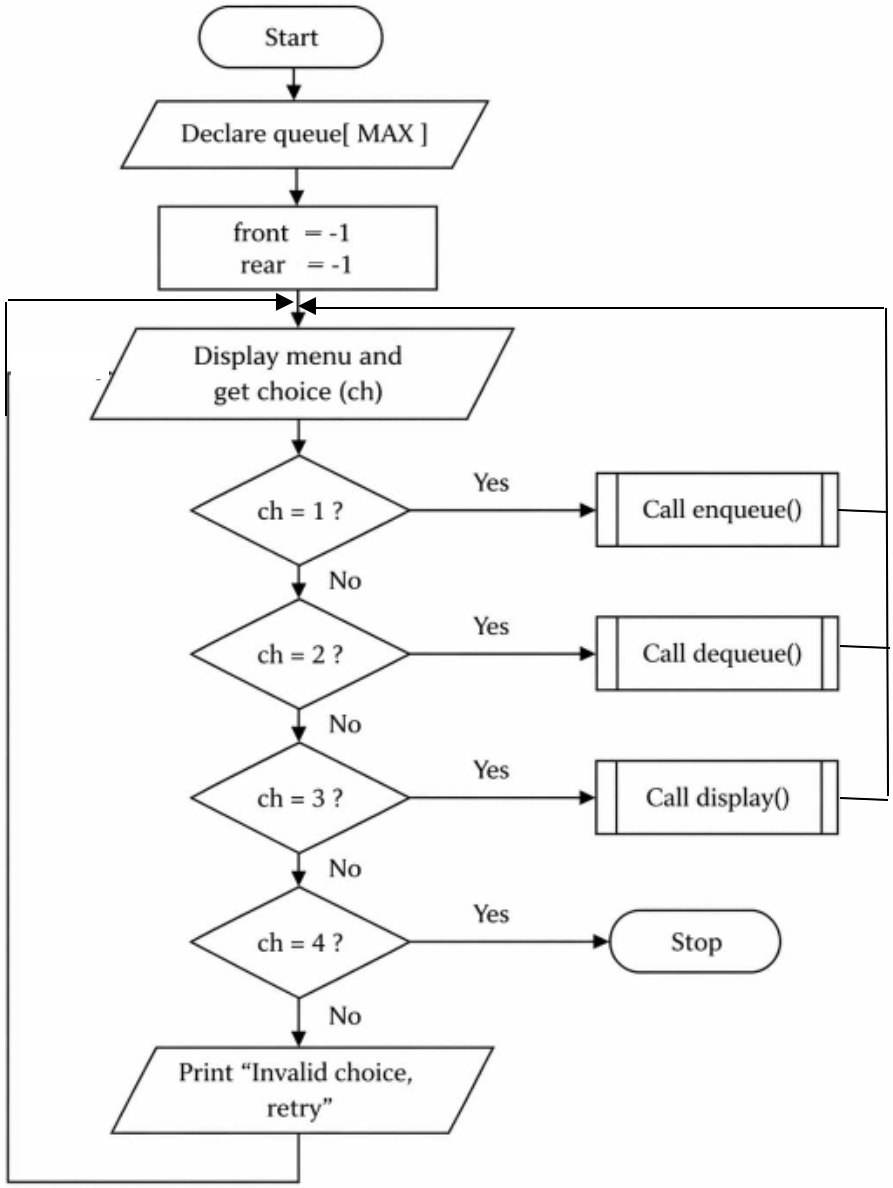
Enqueue( ) Operation Steps (Choice = 1)
Step E1: Check Queue Overflow – If rear == MAX – 1, print “Queue Overflow” and return to Step 4 (Display Main Menu).
Step E2: Read Item to Insert – Read the element to enqueue from the user.
Step E3: Check if Queue Empty – If front == -1, set front = 0.
Step E4: Increment rear – rear = rear + 1.
Step E5: Assign Queue Element – queue[rear] = item.
Step E6: Print Success Message – Print “Element inserted successfully” and return to Step 4 (Display Main Menu).

Dequeue( ) Operation Steps (Choice = 2)
Step D1: Check Queue Underflow – If front == -1 or front > rear, print “Queue Underflow” and return to Step 4(Display Main Menu).
Step D2: Print Deleted Element – Print queue[front].
Step D3: Increment front – front = front + 1 and return to Step 4 (Display Main Menu).

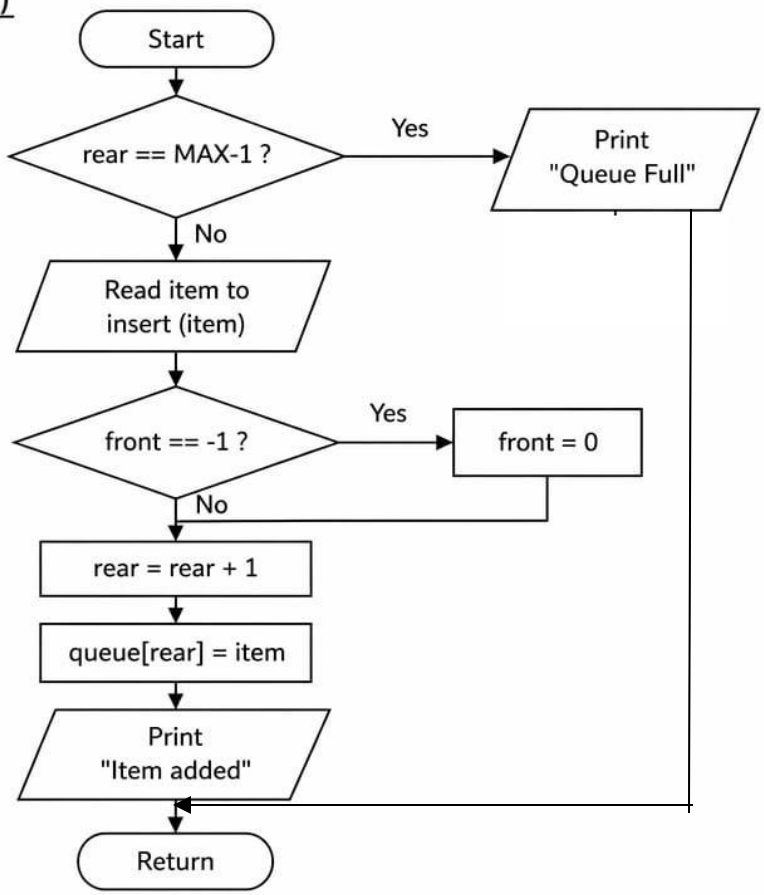
Display( ) Operation Steps (Choice = 3)
Step DS1: Check if Queue is Empty – If front == -1 or front > rear, print “Queue is empty” and return to Step 4(Display Main Menu).
Step DS2: Loop Through Queue – For I = front to rear, print queue[i].
Step DS3: Return to Main Menu – Go back to Step 4 (Display Main Menu).

Exit Operation Steps (Choice = 4)
Step X1: Print Exit Message – Print “Exiting program”.
Step X2: End – Program execution ends.

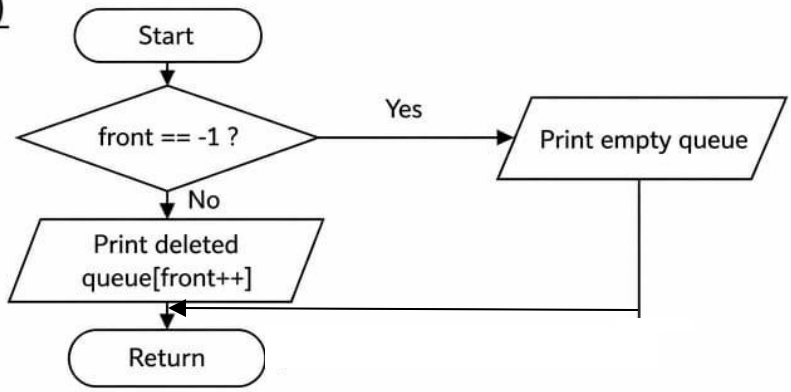
FLOWCHART:


Start
Declare queue[MAX]
front = -1
rear = -1
Display menu and
get choice (ch)
ch = 1 ?
Yes
Call enqueue()
No
ch = 2 ?
Yes
Call dequeue()
No
ch = 3 ?
Yes
Call display()
No
ch = 4 ?
Yes
Stop
No
Print "Invalid choice,
retry"

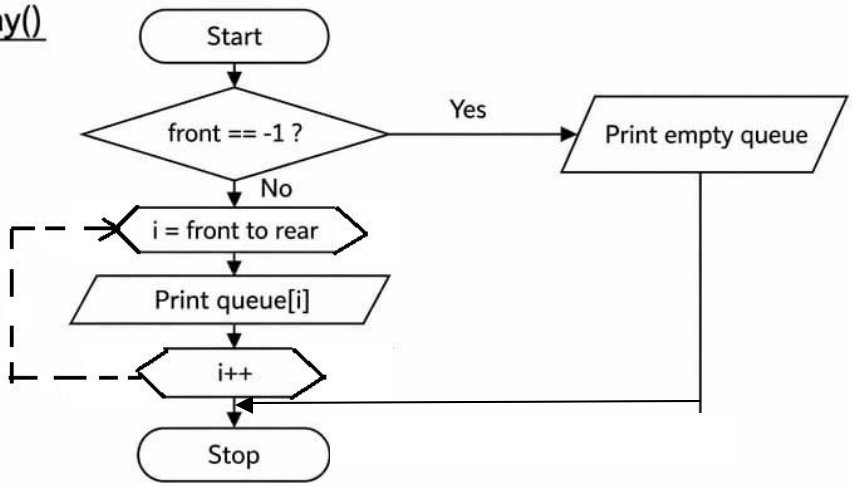
1) enqueue()



2) dequeue()



3) display()



SOURCE CODE:
https://github.com/112534080-stack/DSALab/blob/main/ArrayQueue.c

```
#include < stdio.h>
#define max 5
int queue[max];
int front = -1, rear = -1;
// insert element (enqueue)
void enqueue()
{
    int item;
    if (rear == max – 1)
    {
        printf("queue overflow!\n");
        return;
    }
    printf("enter element to insert: ");
    scanf("% d", &item);
    if (front == -1)
        front = 0;
    queue[++rear] = item;
    printf("element inserted successfully.\n");
}
// delete element (dequeue)
void dequeue()
{
    if (front == -1 || front > rear)
    {
        printf("queue underflow!\n");
        return;
    }
    printf("deleted element: % d\n", queue[front++]);
}
// display queue elements
void display()
{
    if (front == -1 || front > rear)
    {
        printf("queue is empty.\n");
        return;
```

```
    }
    printf("queue elements:\n");
    for (int i = front; i <= rear; i++)
        printf("% d ", queue[i]);
    printf("\n");
}
int main()
{
    int choice;
    do
    {
        printf("\n--- queue menu ---\n");
        printf("1. enqueue\n2. dequeue\n3. display\n4. exit\n");
        printf("enter your choice: ");
        scanf("% d", &choice);
        switch (choice)
        {
        case 1:
            enqueue();
            break;
        case 2:
            dequeue();
            break;
        case 3:
            display();
            break;
        case 4:
            printf("exiting program.\n");
            break;
        default:
            printf("invalid choice!\n");
        }
    } while (choice != 4);
    return 0;
}
```

SAMPLE OUTPUT:

```
--- Queue Menu ---
1. Enqueue
```

```
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter element to insert: 16
Element inserted successfully.

--- Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter element to insert: 23
Element inserted successfully.

--- Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter element to insert: 36
Element inserted successfully.

--- Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Queue elements:
16 23 36

--- Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
```

```
Enter your choice: 3
Queue elements:
16 23 36

--- Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
Deleted element: 16

--- Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Queue elements:
23 36

--- Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 4
Exiting program.
```

8	Program to Implement a Stack using Linked List	27 Apr 2026

AIM:

Program to Implement a Stack using Linked List

ALGORITHM:

1. Initialization
   - Define a structure node with two parts: data (integer) and next (pointer to the next node).
   - Initialize a global pointer top to NULL.
2. Push Operation
   1. Allocate memory for a newnode.

2. If memory allocation fails, print "Heap Overflow" and return.
3. Read the element to be pushed.
4. Set newnode->data = item.
5. Set newnode->next = top (Link the new node to the current top).
6. Update top = newnode (Move the top pointer to the new node).

3. Pop Operation
   1. Check if top == NULL. If yes, print "Stack Underflow" and return.
   2. Create a temporary pointer temp and set temp = top.
   3. Print the data of the element being popped (temp->data).
   4. Update top = top->next (Move the top pointer to the next node).
   5. free(temp) to release the memory.
4. Display Operation
   1. Check if top == NULL. If yes, print "Stack is empty" and return.
   2. Create a temporary pointer temp and set it to top.
   3. While temp is not NULL:
      - Print temp->data.
      - Move temp to the next node (temp = temp->next).
5. Main Menu Loop
   1. Repeat the following until the user chooses to exit (Choice 4):
      - Display options: 1. Push, 2. Pop, 3. Display, 4. Exit.
      - Read user choice.
      - Execute the corresponding function based on choice.

SOURCE CODE:

https://github.com/112534080-stack/DSALab/blob/main/StacklinkedList.c

```
#include <stdio.h>
#include <stdlib.h>
// structure for stack node
struct node
{
    int data;
    struct node *next;
};
struct node *top = null;
// push operation
void push()
{
    int item;
    struct node *newnode;
    newnode = (struct node *)malloc(sizeof(struct node));
    if (newnode == null)
```

```
    {
        printf("heap overflow!\n");
        return;
    }
    prints("enter element to push: ");
    scanf("% d", &item);
    newnode->data = item;
    newnode->next = top;
    top = newnode;
    printf("element pushed successfully.\n");
}
// pop operation
void pop()
{
    struct node *temp;
    if (top == null)
    {
        printf("stack underflow!\n");
        return;
    }
    temp = top;
    printf("popped element: % d\n", temp->data);
    top = temp->next;
    free(temp);
}
// display stack
void display()
{
    struct node *temp;
    if (top == null)
    {
        printf("stack is empty.\n");
        return;
    }
    printf("stack elements:\n");
    temp = top;
    while (temp != null)
    {
        printf("% d\n", temp->data);
```

```
        temp = temp->next;
    }
}
int main()
{
    int choice;
    do
    {
        printf("\n--- stack menu ---\n");
        printf("1. push\n2. pop\n3. display\n4. exit\n");
        printf("enter your choice: ");
        scanf("% d", &choice);
        switch (choice)
        {
        case 1:
            push();
            break;
        case 2:
            pop();
            break;
        case 3:
            display();
            break;
        case 4:
            printf("exiting program.\n");
            break;
        default:
            printf("invalid choice!\n");
        }
    } while (choice != 4);
    return 0;
}
```

SAMPLE OUTPUT:

```
--- Stack Menu ---
1. Push
2. Pop
3. Display
```

```
4. Exit
Enter your choice: 1
Enter element to push: 23
Element pushed successfully.

--- Stack Menu ---
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter element to push: 35
Element pushed successfully.

--- Stack Menu ---
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter element to push: 36
Element pushed successfully.

--- Stack Menu ---
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Stack elements:
36
35
23

--- Stack Menu ---
1. Push
2. Pop
3. Display
4. Exit
```

```
Enter your choice: 3
Stack elements:
36
35
23

--- Stack Menu ---
1. Push
2. Pop
3. Display
4. Exit
Enter your choice:
2
Popped element: 36

--- Stack Menu ---
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Stack elements:
35
23

--- Stack Menu ---
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 4
Exiting program.
```

9	Program to Implement a Queue using Linked List	03 Apr 2026

AIM:

Program to Implement a Queue using Linked List

ALGORITHM:

1. Initialization
   - Define a structure node with data and a pointer next.
   - Initialize two global pointers: front = NULL and rear = NULL.
2. Enqueue Operation (Insertion)
   1. Allocate memory for a newNode.
   2. If memory allocation fails, print "Heap Overflow" and exit the function.
   3. Read the value to be inserted into newNode->data.
   4. Set newNode->next = NULL.
   5. Check if Queue is Empty: If rear == NULL:
      - Set both front and rear to newNode.
   6. Else (Queue has elements):
      - Link the current rear's next to the new node: rear->next = newNode.
      - Update the rear pointer: rear = newNode.
   7. Print "Element inserted successfully."
3. Dequeue Operation (Deletion)
   1. Check if Queue is Empty: If front == NULL:
      - Print "Queue Underflow" and return.
   2. Create a temporary pointer temp and set temp = front.
   3. Print the data of the node being deleted (temp->data).
   4. Move the front pointer to the next node: front = front->next.
   5. Check if Queue became empty: If front == NULL, set rear = NULL.
   6. Free the memory occupied by temp.
4. Display Operation
   1. Check if front == NULL. If yes, print "Queue is empty."
   2. Initialize a pointer temp to front.
   3. While temp != NULL:
      - Print temp->data.
      - Move temp to the next node: temp = temp->next.
5. Main Menu
   1. Display choices: 1. Enqueue, 2. Dequeue, 3. Display, 4. Exit.
   2. Perform the selected operation based on user input.
   3. Repeat until the user chooses to exit.

SOURCE CODE;

https://github.com/112534080-stack/DSALab/blob/main/Queuelinkedlist.c

```
#include <stdio.h>
```

```
#include < stdlib.h>
// structure for queue node
struct node
{
    int data;
    struct node *next;
};
struct node *front = null;
struct node *rear = null;
// enqueue operation
void enqueue()
{
    int item;
    struct node *newnode;
    newnode = (struct node *)malloc(sizeof(struct node));
    if (newnode == null)
    {
        printf("heap overflow!\n");
        return;
    }
    printf("enter element to insert: ");
    scanf("% d", &item);
    newnode->data = item;
    newnode->next = null;
    if (rear == null)
    {
        front = rear = newnode;
    }
    else
    {
        rear->next = newnode;
        rear = newnode;
    }
    printf("element inserted successfully.\n");
}
// dequeue operation
void dequeue()
{
    struct node *temp;
    if (front == null)
    {
        printf("queue underflow!\n");
        return;
```

```
    }
    temp = front;
    printf("deleted element: % d\n", temp->data);
    front = front->next;
    if (front == null)
    rear = null;
    free(temp);
}
// display queue elements
void display()
{
    struct node *temp;
    if (front == null)
    {
        printf("queue is empty.\n");
        return;
    }
    printf("queue elements:\n");
    temp = front;
    while (temp != null)
    {
        printf("% d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
int main()
{
    int choice;
    do
    {
        printf("\n--- queue menu ---\n");
        printf("1. enqueue\n2. dequeue\n3. display\n4. exit\n");
        printf("enter your choice: ");
        scanf("% d", &choice);
        switch (choice)
        {
        case 1:
            enqueue();
            break;
        case 2:
            dequeue();
            break;
```

```
        case 3:
            display();
            break;
        case 4:
            printf("exiting program.\n");
            break;
        default:
            printf("invalid choice!\n");
        }
    } while (choice != 4);

    return 0;
}
```

SAMPLE OUTPUT;

```
--- Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter element to insert: 12
Element inserted successfully.

--- Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter element to insert: 51
Element inserted successfully.

--- Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Queue elements:
12 51
```

```
--- Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
Deleted element: 12

--- Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Queue elements:
51

--- Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 4
Exiting program.
```